

Annealing-Based Bayesian Networks for Cardinality Estimation

Matea Qazolli

Universität Passau
Chair of Scalable Database Systems
Supervisor: Prof. Stefanie Scherzinger

Master's Thesis Defense, 25 June 2026

Outline

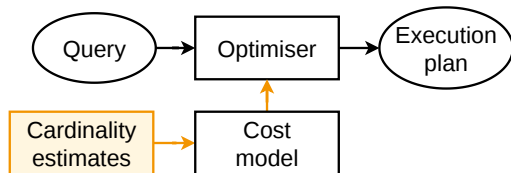
1. Motivation
2. Background
3. Research Questions
4. Methodology
5. Experimental Evaluation
6. Conclusion

Cardinality estimation

- Estimates how many tuples a query will return.

Optimiser

- Chooses an execution plan before the query runs.
- Plan costs depend on cardinality estimates.



Source: Adapted from Halford et al. (2020), Slide 10 [5].

Failure mode

- Correlations → wrong estimates
→ wrong costs → bad plans.

**Focus: dependency-aware
cardinality estimation.**

Main idea [1]

- ▶ Estimate query result sizes from compact table statistics.
- ▶ Examples: table size, distinct values, most-common values, and histograms.

Cardinality estimation: $\hat{C}(q) = N \cdot \hat{\sigma}(q)$

N : relation size.

$\hat{\sigma}(q)$: estimated fraction of matching tuples.

Independence assumption: predicates A and B are treated as independent.

$$\hat{\sigma}(A \wedge B) \approx \hat{\sigma}(A) \cdot \hat{\sigma}(B)$$

PostgreSQL baseline [2]

- ▶ ANALYZE collects planner statistics;
EXPLAIN reports estimated row counts.

$q : \text{MAKE} = \text{Mazda} \wedge \text{MODEL} = \text{CX-5} \wedge \text{COLOR} = \text{white}$

Statistics

Attr.	Value	Distinct
MAKE	Mazda	10
MODEL	CX-5	100
COLOR	white	10

MODEL = CX-5 already implies
MAKE = Mazda.

Independence assumption

$$\hat{\sigma}_{\text{ind}} = 10^{-1} \cdot 10^{-2} \cdot 10^{-1} = 10^{-4}$$

Dependency-aware estimate

$$\hat{\sigma}_{\text{dep}} \approx 10^{-2} \cdot 10^{-1} = 10^{-3}$$

Result: tenfold underestimation.

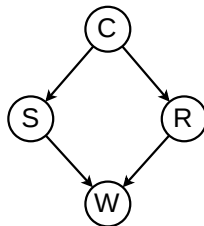
Example adapted from Markl et al. (2005) [12].

- ▶ Graph structure encodes conditional independencies [3].
- ▶ CPTs define local conditional probabilities.

Variable	States
C : Cloud	t/f
S : Sprinkler	on/off
R : Rain	t/f
W : WetGrass	t/f

t : true f : false

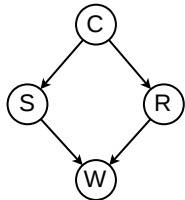
Ground truth structure



Joint distribution factorisation:

$$P(C, S, R, W) = P(C) P(S \mid C) P(R \mid C) P(W \mid S, R)$$

Ground truth structure



Conditional probability tables (CPTs)

$P(C)$

$C = t$	$C = f$
0.3	0.7

$P(S | C)$

C	$S = \text{on}$	$S = \text{off}$
t	0.2	0.8
f	0.8	0.2

$P(R | C)$

C	$R = t$	$R = f$
t	0.6	0.4
f	0.3	0.7

$P(W | S, R)$

S	R	$W = t$	$W = f$
on	t	1.0	0.0
on	f	1.0	0.0
off	t	1.0	0.0
off	f	0.1	0.9

Main idea [4]

- ▶ Query predicates define an event over the BN variables.
- ▶ The fitted BN estimates the probability of this event.

WetGrass example

$$q: S = \text{on} \wedge W = t$$

$$\hat{C}_{\text{BN}}(q) = N \cdot \hat{P}_{\text{BN}}(S = \text{on}, W = t)$$

Chow–Liu baseline [6]

- ▶ Learns a tree-structured BN using pairwise mutual information.
- ▶ Restricts each variable to at most one parent.

QUBO idea [9]

$$\min_{x \in \{0,1\}^n} x^T Q x$$

x : binary decisions.

Q : matrix storing scores and interactions

- ▶ Lower objective value means a better encoded solution.

QUBO-BNSL idea [7, 8]

- ▶ BN structure learning is formulated as a binary optimisation problem.
- ▶ The objective balances data-fit scores with structural penalties.

Annealing principle

- ▶ Annealing searches over binary assignments of the QUBO variables.
- ▶ Each assignment represents one candidate BN structure.
- ▶ One **read** is one sampled low-energy assignment, which is decoded into an adjacency matrix.

Solvers used

- ▶ **SA**: simulated annealing; classical stochastic search.
- ▶ **SQA**: simulated quantum annealing; quantum-inspired stochastic search.

1. **Integration:** Can QUBO-based Bayesian-network structure learning be integrated into an end-to-end cardinality-estimation workflow?
2. **Accuracy:** Do richer learned Bayesian-network structures improve query-level cardinality estimates?

1. **Representation:** tabular view $\rightarrow$ solver view.
2. **Structure learning:** QUBO-BNSL $\rightarrow$ solver output.
3. **Decoding:** solver output $\rightarrow$ adjacency matrix.
4. **Parameter fitting:** BN structure $\rightarrow$ fitted BN.
5. **Query inference:** selection query $\rightarrow$ cardinality estimate.

Step 1: Representation

Tabular view

- ▶ Original attributes and values.
- ▶ Used for parameter fitting.
- ▶ Used for query evaluation.

C	S	R	W
t	on	t	t
t	off	t	f
f	on	f	t

Solver view

- ▶ Finite integer states.
- ▶ Used for QUBO construction.
- ▶ Used for structure learning.

C	S	R	W
1	1	1	1
1	0	1	0
0	1	0	1

Both views must stay aligned through the same variable order.

Input

- ▶ Solver-view observations.
- ▶ Number of variables.
- ▶ State counts per variable.

QUBO construction

- ▶ Compute local scores.
- ▶ Add structural penalties.
- ▶ Restrict parent sets to $m = 2$.

QUBO solving

- ▶ Solve with SA/SQA.
- ▶ Return low-energy assignment.

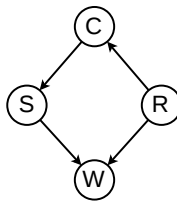
Goal: formulate BNSL as a QUBO optimisation problem and solve it.

Adjacency matrix

	<i>C</i>	<i>S</i>	<i>R</i>	<i>W</i>
<i>C</i>	0	1	0	0
<i>S</i>	0	0	0	1
<i>R</i>	1	0	0	1
<i>W</i>	0	0	0	0

- ▶ Decoded from low-energy assignment.
- ▶ 1: edge from row to column.

Corresponding Bayesian network



- ▶ Decoded structure is mapped back to the tabular view.

Output: a Bayesian-network structure ready for fitting.

Fitting logic

- ▶ Parent sets are fixed by the graph.
- ▶ CPT values are estimated from tuple counts.

Maximum-likelihood estimate

$$\hat{P}(X = x \mid \text{Pa}(X) = p) = \frac{n(X = x, \text{Pa}(X) = p)}{n(\text{Pa}(X) = p)}$$

$n(\cdot)$ = number of matching tuples

WetGrass example:

$$\hat{P}(W = t \mid S = \text{on}, R = t) = \frac{n(W = t, S = \text{on}, R = t)}{n(S = \text{on}, R = t)}$$

Output: fitted BN with learned CPTs.

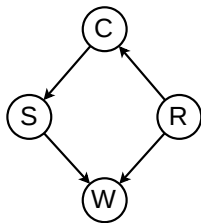
- ▶ Query predicates define the event to estimate.
- ▶ The fitted BN estimates the probability of this event.
- ▶ This probability is the **estimated selectivity**.
- ▶ **Variable elimination** marginalises non-query variables.
- ▶ **Cardinality** is selectivity times relation size.

Output: query-level cardinality estimate.

- ▶ All variables are fixed, so inference multiplies the fitted CPT entries.

$$q_1 : R = t \wedge C = t \wedge S = \text{on} \wedge W = t$$

$$\begin{aligned}\hat{\sigma}(q_1) &= \hat{P}(R = t) \hat{P}(C = t \mid R = t) \\ &\quad \cdot \hat{P}(S = \text{on} \mid C = t) \hat{P}(W = t \mid S = \text{on}, R = t).\end{aligned}$$

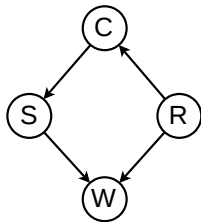


Step 5: Inference Case 2

- Query fixes S and W , while R and C are summed out.

$q_2 : S = \text{on} \wedge W = t$ hidden: R, C

$$\begin{aligned}\hat{o}(q_2) = & \sum_{r \in \{t, f\}} \sum_{c \in \{t, f\}} \hat{P}(R = r) \hat{P}(C = c \mid R = r) \\ & \cdot \hat{P}(S = \text{on} \mid C = c) \hat{P}(W = t \mid S = \text{on}, R = r).\end{aligned}$$



$$q(y, \hat{y}) = \frac{\max(y, \hat{y})}{\min(y, \hat{y})}$$

- ▶ Standard metric for cardinality-estimation error [10].
- ▶ $q = 1$: exact estimate.
- ▶ Symmetric for overestimation and underestimation.
- ▶ Measures multiplicative error.
- ▶ Direct factor-based comparison between different estimators.

- ▶ Synthetic relation generated from WetGrass Bayesian network.
- ▶ Binary categorical attributes: *C* (Cloud), *S* (Sprinkler), *R* (Rain), *W* (WetGrass).
- ▶ 10,000 tuples and no missing values.

Query	SA	SQA	Classical	PostgreSQL
Q1	1.000	1.000	1.000	1.000
Q2	1.000	1.000	1.126	1.159
Q3	1.000	1.000	1.000	1.212
Q4	1.000	1.000	1.272	1.100
Q5	1.000	1.000	1.319	4.729
Q6	1.000	1.000	1.276	7.140

Table 1. Per-query q-error values for the six WetGrass workload queries using 100 reads and 20 trials. Lower values indicate more accurate cardinality estimates.

- ▶ NHANES Age Prediction subset [11]; 2,278 tuples and no missing values.
- ▶ Attributes: age_group, BMXBMI, RIAGENDR, and DIQ010.

Query	SA	SQA	Classical	PostgreSQL
Q1	1.000	1.000	1.000	1.000
Q2	1.000	1.000	1.000	1.286
Q3	1.000	1.000	1.166	4.000
Q4	1.000	1.000	1.027	1.063
Q5	1.000	1.000	1.175	1.600
Q6	1.008	1.008	1.276	1.415

Table 2. Per-query q-error values for the six NHANES workload queries using 100 reads and 10 trials. Lower values indicate more accurate cardinality estimates.

- ▶ Synthetic transaction relation based on market-basket item co-occurrence patterns.
- ▶ Binary item attributes: Beer, Bread, Cola, Diapers, Eggs, and Milk.
- ▶ 10,000 tuples generated from five base transaction templates.

Query	SA	SQA	Classical	PostgreSQL
Q1	1.0002	1.0002	1.0000	1.0000
Q2	1.0000	1.0000	1.0000	1.2575
Q3	1.0002	1.0002	1.0000	1.0696
Q4	1.0000	1.0000	1.0000	4.9655
Q5	1.0006	1.0006	1.1233	1.2809
Q6	1.0007	1.0007	2.0299	8.3750

Table 3. Per-query q-error values for the six Market Basket workload queries using 100 reads and 20 trials. Lower values indicate more accurate cardinality estimates.

1. **Integration:** ANNEALBN-CE forms a feasible end-to-end workflow for query-level cardinality estimation.
2. **Accuracy:** Accuracy gains are conditional and strongest for dependency-sensitive multi-predicate queries.

- ▶ **Single-relation scope:** selection predicates only; no joins.
- ▶ **Scalability:** QUBO size grows quickly with variables, states, and candidate parent sets.
- ▶ **Discretisation:** numerical attributes must be converted into finite states.
- ▶ **Solver scope:** no physical quantum hardware.
- ▶ **Evaluation level:** query-plan and runtime effects are not measured.

- ▶ **Richer query workloads:** extend beyond equality selections to joins, range predicates, and pattern matching.
- ▶ **Numerical representation:** study discretisation strategies for continuous attributes before structure learning.
- ▶ **Query-scoped models:** learn smaller BNs over query-relevant attributes instead of one global structure.
- ▶ **Real-engine integration:** investigate how AnnealBN-CE could fit into a production query engine.

Thank you

Questions?

References

- [1] Lan, H., Bao, Z., and Peng, Y. (2021).
A survey on advancing the DBMS query optimizer: cardinality estimation, cost model, and plan enumeration.
Data Science and Engineering, 6, 86–101.
- [2] PostgreSQL Global Development Group.
<https://www.postgresql.org/docs/current/planner-stats.html>
<https://www.postgresql.org/docs/current/row-estimation-examples.html>
- [3] Jensen, F. V., and Nielsen, T. D. (2007).
Bayesian Networks and Decision Graphs.
Springer, 2nd edition.
- [4] Halford, M., Saint-Pierre, P., and Morvan, F. (2019).
An approach based on Bayesian networks for query selectivity estimation.
In *Database Systems for Advanced Applications (DASFAA 2019)*, LNCS 11447, 3–19.
- [5] Halford, M., Saint-Pierre, P., and Morvan, F. (2020).
Machine learning for query selectivity estimation in relational databases.
PhD defence presentation slides.
https://maxhalford.github.io/files/PhD_presentation.pdf

References

- [6] Chow, C. K., and Liu, C. N. (1968).
Approximating discrete probability distributions with dependence trees.
IEEE Transactions on Information Theory, 14(3), 462–467.
- [7] O’Gorman, B., Babbush, R., Perdomo-Ortiz, A., Aspuru-Guzik, A., and Smelyanskiy, V. (2015).
Bayesian network structure learning using quantum annealing.
The European Physical Journal Special Topics, 224(1), 163–188.
- [8] Zardini, E., Rizzoli, M., Dissegna, S., Blanzieri, E., and Pastorello, D. (2022).
Reconstructing Bayesian networks on a quantum annealer.
Quantum Information and Computation, 22(15–16), 1320–1350.
- [9] Glover, F., Kochenberger, G., and Du, Y. (2019).
Quantum Bridge Analytics I: A tutorial on formulating and using QUBO models.
4OR, 17, 335–371.
- [10] Han, Y., Wu, Z., Wu, P., Zhu, R., Yang, J., Tan, L. W., Zeng, K., Cong, G., Qin, Y., Pfadler, A., Qian, Z., Zhou, J., Li, J., and Cui, B. (2022).
Cardinality estimation in DBMS: A comprehensive benchmark evaluation.
Proceedings of the VLDB Endowment, 15(4), 752–765.

References

- [11] [UCI Machine Learning Repository](#). (2019).
National Health and Nutrition Health Survey 2013–2014 (NHANES) Age Prediction Subset.
<https://doi.org/10.24432/C5BS66>
- [12] [Markl, V., Megiddo, N., Kutsch, M., Tran, T. M., Haas, P. J., and Srivastava, U.](#) (2005).
Consistently estimating the selectivity of conjuncts of predicates.
In Proceedings of the 31st International Conference on Very Large Data Bases, 373–384.

- ▶ **Compute pairwise MI:** score every pair of attributes by statistical dependency.
- ▶ **Build complete graph:** use the MI scores as edge weights.
- ▶ **Extract maximum spanning tree:** keep the strongest pairwise dependency edges.
- ▶ **Orient the tree:** choose a root and obtain a tree-structured Bayesian network.

$$MI(X, Y) = \sum_{x,y} P(x, y) \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- ▶ X, Y : two discrete attributes whose dependency is measured.
- ▶ $P(x, y)$: how often the two values occur together in the data.
- ▶ $P(x)$ and $P(y)$: how often each value occurs on its own in the data.
- ▶ $P(x)P(y)$: how often the two values would occur together if the attributes were independent.
- ▶ $MI(X, Y) = 0$ indicates independence.

QUBO optimisation problem

$$\min_{x \in \{0,1\}^n} y = x^T Q x$$

- ▶ x : binary decision vector.
- ▶ Q : square matrix containing linear terms and pairwise interactions.
- ▶ Maximisation objectives can be converted to minimisation by changing the sign.

Matrix structure

- ▶ **Diagonal entries** q_{ii} : linear bias for activating decision variable x_i .
- ▶ **Off-diagonal entries** q_{ij} : interaction between two binary decisions x_i and x_j .

$$H = H_{\text{score}} + H_{\text{max}} + H_{\text{cycle}}$$

- ▶ **Arc variables:** $d_{ji} = 1$ means that the edge $X_j \rightarrow X_i$ is selected.
- ▶ **Local scores:** candidate parent sets are scored from frequency counts in the data.
- ▶ H_{score} : favours parent sets that fit the observed data well.
- ▶ H_{max} : enforces the maximum in-degree restriction, here $m = 2$.
- ▶ H_{cycle} : enforces acyclicity, so the result is a valid Bayesian-network structure.

Lower energy corresponds to a better-scoring valid Bayesian network.

SA

- ▶ Classical stochastic search.
- ▶ Uses a temperature schedule to move from exploration to exploitation.
- ▶ Can accept higher-energy states early to escape local minima.

SQA

- ▶ Quantum-inspired stochastic search.
- ▶ Simulates quantum fluctuations using Trotter replicas.
- ▶ Samples low-energy states through a classical approximation of quantum annealing.

Query	True	SA	SQA	Classical	PostgreSQL
Q1	1,746	1,746	1,746	1,746.00	1,746
Q2	2,136	2,136	2,136	1,897.26	2,476
Q3	3,900	3,900	3,900	3,900.00	3,219
Q4	4,160	4,160	4,160	3,270.51	3,782
Q5	240	240	240	316.50	1,135
Q6	864	864	864	1,102.74	121

Estimator	Min q	Median q	Max q
SA	1.000	1.000	1.000
SQA	1.000	1.000	1.000
Classical baseline	1.000	1.199	1.319
PostgreSQL	1.000	1.185	7.140

SA and SQA match all true cardinalities; baseline errors appear on dependency-sensitive queries.

Query	True	SA	SQA	Classical	PostgreSQL
Q1	2199	2199	2199	2199.00	2199
Q2	9	9	9	9.00	7
Q3	4	4	4	3.43	1
Q4	675	675	675	657.57	635
Q5	8	8	8	6.81	5
Q6	75	75.6	75.6	58.78	53

Estimator	Min q	Median q	Max q
SA	1.000	1.000	1.008
SQA	1.000	1.000	1.008
Classical baseline	1.000	1.096	1.276
PostgreSQL	1.000	1.350	4.000

SA and SQA stayed near-exact across all six queries.

Market Basket Results: Binary Transaction

Query	True	SA	SQA	Classical	PostgreSQL
Q1	8,057	8,055.72	8,055.72	8,057.00	8,057
Q2	5,934	5,934.00	5,934.00	5,934.00	4,719
Q3	5,937	5,935.72	5,935.72	5,937.00	6,350
Q4	2,016	2,016.00	2,016.00	2,016.00	406
Q5	3,994	3,991.44	3,991.44	4,486.53	5,116
Q6	1,943	1,944.28	1,944.28	957.21	232

Estimator	Min q	Median q	Max q
SA	1.0000	1.0002	1.0007
SQA	1.0000	1.0002	1.0007
Classical baseline	1.0000	1.0000	2.0299
PostgreSQL	1.0000	1.2692	8.3750

SA and SQA remain near-exact, while the baselines show larger tail errors.

Query	Selection predicate
Q1	wetgrass = 'f'
Q2	cloud = 't' AND wetgrass = 't'
Q3	rain = 't' AND wetgrass = 't'
Q4	sprinkler = 'on' AND rain = 'f'
Q5	cloud = 't' AND sprinkler = 'on' AND rain = 'f'
Q6	cloud = 't' AND sprinkler = 'off' AND rain = 'f' AND wetgrass = 'f'

Query	Selection predicate
Q1	DIQ010 = 2
Q2	BMXBMI >= 30.0 AND DIQ010 = 1
Q3	age_group = 'Senior' AND BMXBMI >= 30.0 AND DIQ010 = 1
Q4	age_group = 'Adult' AND BMXBMI >= 18.5 AND BMXBMI < 25.0 AND DIQ010 = 2
Q5	age_group = 'Adult' AND BMXBMI >= 25.0 AND BMXBMI < 30.0 AND DIQ010 = 1
Q6	age_group = 'Senior' AND RIAGENDR = 1 AND BMXBMI >= 25.0 AND BMXBMI < 30.0 AND DIQ010 = 2

Query	Selection predicate
Q1	Bread = 1
Q2	Beer = 1 AND Diapers = 1
Q3	Milk = 1 AND Diapers = 1
Q4	Eggs = 1 AND Milk = 0
Q5	Bread = 1 AND Diapers = 1 AND Milk = 1
Q6	Beer = 1 AND Bread = 0 AND Cola = 1 AND Diapers = 1 AND Eggs = 0 AND Milk = 1

Attribute	Prepared values	Solver states
age_group	Adult, Senior	0, 1
BMXBMI	< 18.5 , $[18.5, 25)$, $[25, 30)$, ≥ 30	0, 1, 2, 3
RIAGENDR	1.0, 2.0	0, 1
DIQ010	1.0, 2.0, 3.0	0, 1, 2

BMXBMI is discretised into BMI intervals; all retained attributes are then represented as finite integer states for QUBO construction.

Structure source	Structures	Distinct CE	Median q	Max q	Exact
SA, $R = 250$, $T = 30$	11	1	1.04	1.16	33.33%
SQA, $R = 100$, $T = 30$	18	1	1.04	1.16	33.33%
Random structures	960	838	1.14	18.21	25.28%

The robustness check shows that the learned annealing structures are more stable and reliable than random admissible structures.